EEEN30160 – Biomedical Signal Processing

Assignment 3

Oisín Hogan

19355123

Contents

Problem 1: 3
Code: 3
Output 4
Discussion 5
a) 5
b) 5
c) 5
Problem 2: 6
Code: 6
Outputs: 7
Discussion: 7
a) 7
b) 7
c) 7
d) 7
Problem 3: 8
Code: 8
Output: 9
Discussion 9
Problem 4: 10
Code: 10
Output: 11
Discussion 11

Problem 1:

Code:

```
clc
close all
clear
%clear all values, plots and command window

%a)
a = [1 .125 -1/2]; %observe a's from equation
b = [0, 2, 3]; % observe b's from equation

[z, p, k] = tf2zpk(b, a); %poles and zeros of system

zplane(z,p) %plot poles and zeros of system
title('Z-Plane')
%b)

%c)
x = rand(1000,1); % generate random vector x
xf = filtfilt(b, a, x); % filt x with filter

figure
tiledlayout(2,1)

nexttile% plot amplitude of unfiltered x
plot(x)
title('X( Unfiltered)')
xlabel('time')
ylabel('Amplitude')
nexttile%plot amplitude of filtered x
plot(xf, 'r')
title('X (Filtered)')
xlabel('time')
ylabel('Amplitude')

fdom = (0: 1/1000: 1-1/1000);% create frequency domain for x
fxf = abs(fft(xf)); % fourrier transform of filtered x
fxf = (fftshift(fxf)); % find phase of filtered x
fx = abs(fft(x)); % fourrier transform of x
fx = fftshift(fx);% find phase of x
figure
tiledlayout(2,1)
nexttile
plot(fdom, fx);
title('X')
xlabel('Frequency')
ylabel('Amplitude')
nexttile
plot(fdom, fxf)
title('X (Filtered)')
xlabel('Frequency')
ylabel('Amplitude')
```

Output

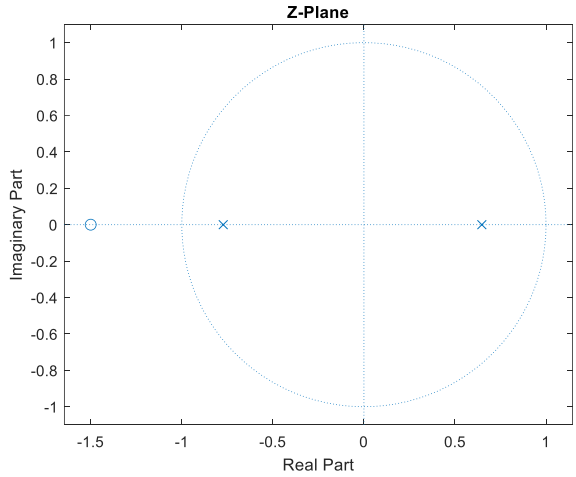

Figure 1

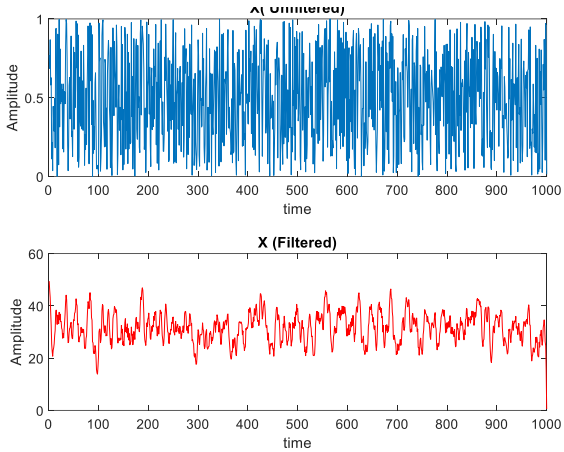

Figure 2

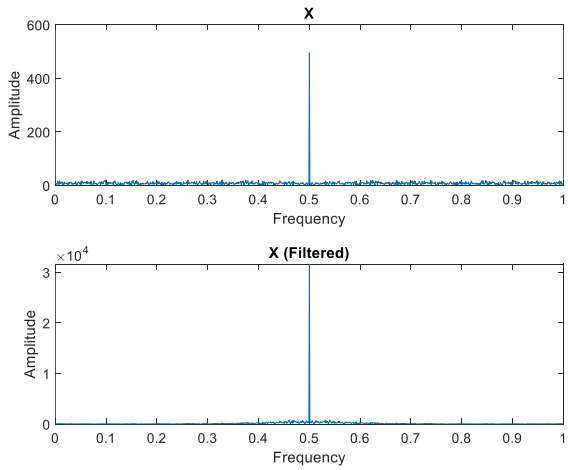

Figure 3

Discussion

a)

From observing the transfer function, the constants were found to be:

$$y(n) = 2x(n-1) – 3x(n-2) – 0.1250\, y(n-1) + 0.5\, y(n-2)$$

	A	B
0	1	0
1	0.125	2
2	-0.5	3

Table 1

b)

This system is not stable, as can be seen in the z-plane graph in figure 1, the poles lie outside the circle

c)

This filter has amplified the signal x

Problem 2:

Code:

```
clc
close all
clear
%clear all values, plots and command window

%a)
N=100; % N = 100 for filter
A = [1, -.2, 0,  .1]; % define A according to transfer function
B = [1, .1, -.3]; % define B according to transfer function
[z, p, k] = tf2zpk(B, A); % find zeros and poles of transfer function
figure% plot the filters and poles of this function
zplane(z,p)
title('Z-Plane')

%b)
freqz(B, A, N);% plot the magnitude of hte filter response for the transfer
function
title('Magnitude of Filter Response')
%c)

%d)
x = rand(1000,1);% random 1000 long signal
figure
tiledlayout(2,1)
nexttile% plot x
plot(x)
title('X signal')
xlabel('Samples')
ylabel('Amplitude')
nexttile % filter x and plot the filtered version
xf = filtfilt(B,A,x);
plot(xf)
title('Filtered X signal')
xlabel('Samples')
ylabel('Amplitude')

figure
tiledlayout(2,1)
nexttile
fdom = (0: 1/1000: 1-1/1000);% frequency domain array created
fxf = abs(fft(xf)); % fourrier transform of filtered x
fxf = (fftshift(fxf)); % phase shift of fft(filtered x)
fx = abs(fft(x)); % fourrier transform of x
fx = fftshift(fx);% phase shift of fft(x)
plot(fdom, fx); % plot phase shifted fft(x)
title('Filtered X signal')
xlabel('Frequencies')
ylabel('Amplitude')
nexttile
plot(fdom, fxf)% plot phase shifted fft(filtered x)
title('Filtered X signal')
xlabel('Frequencies')
ylabel('Amplitude')
```

Outputs:

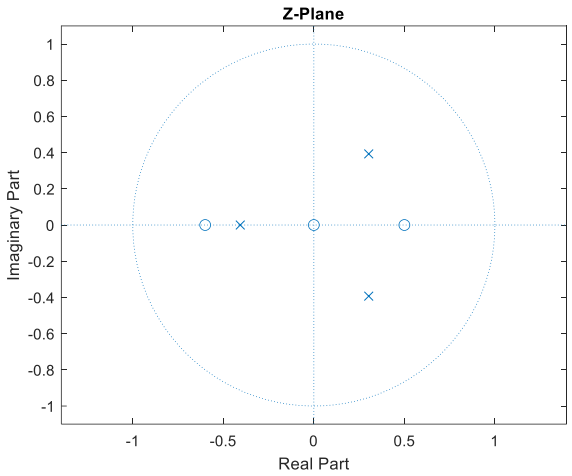

Figure 4

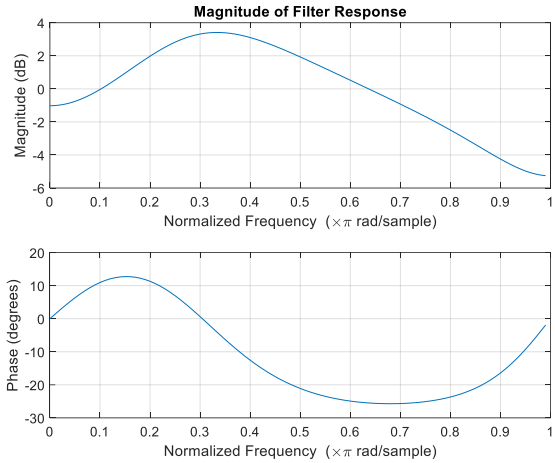

Figure 6

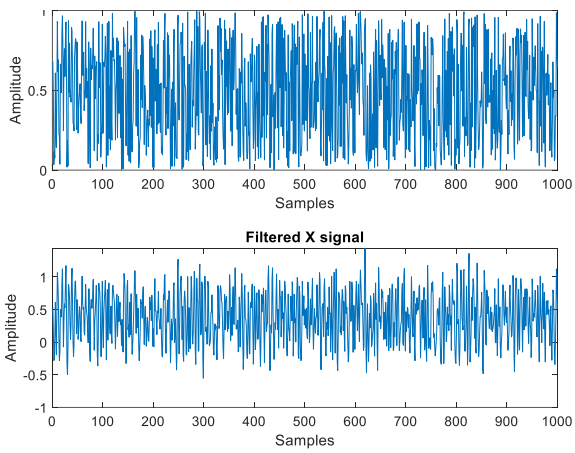

Figure 7

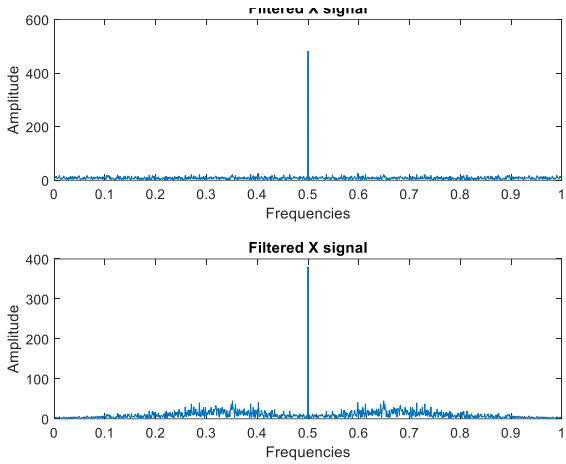

Figure 5

Discussion:

a)

As can be seen in figure 4, all the zeros and poles lie inside the circle when plotted using the zplane() function. This meant that the filter was stable.

b)

The plot for the frequency can be seen in figure 5.

c)

This is a band pass filter, as can be seen in figure 5, where frequencies above and below about 0.35 of the normalised frequency were reduced.

d)

From figure 7, it can be seen that the filter had no phase delay associated with it, which can be seen by the similarities of the 2 graphs.

Problem 3:

Code:

```
clc
close all
clear
%clear all values, plots and command window

%a)
fs = pi; % sampling frequency is pi Hz
Wn = 1/4; % the 1/8 pi is equal to 1/4 of pi/2
Nf = 12; % the order of the FIR filter is 12th
Bf = fir1(Nf, Wn); %Lowpass FIR filter

%b)
Nb = 3; % order of the butterworth filter is 3
[Bb, Ab] = butter(Nb, Wn); %Lowpass Buttereworth filter

%c)
figure
N= 100; % plot the fitler responses for N = 100
[Hf, Wf] = freqz(Bf, 1, N); % freqz for the FIR filter
[Hb, Wb] = freqz(Bb, Ab, N); % freqz for the butterworth filter

angf = angle(Hf); % find phase of Hf
angb = angle(Hb); % find phase of Hb

tiledlayout(2,1)
nexttile% plot the magnitude of Hf and Hb
fdom = (0: 1/100: 1-1/100);
plot(fdom , abs(Hf))
title('Response Magnitude of Filter')
xlabel('Normalised Frequency')
ylabel('Amplitude')
hold on
plot(fdom ,abs(Hb))
legend('FIR', 'Butterworth')

nexttile% plot the phase of Hf and Hb
plot(abs(angf))
title('Phase Response of Filter')
xlabel('Normalised Frequency')
ylabel('Amplitude')
hold on
plot(abs(angb))
legend('FIR', 'Butterworth')
```

Output:

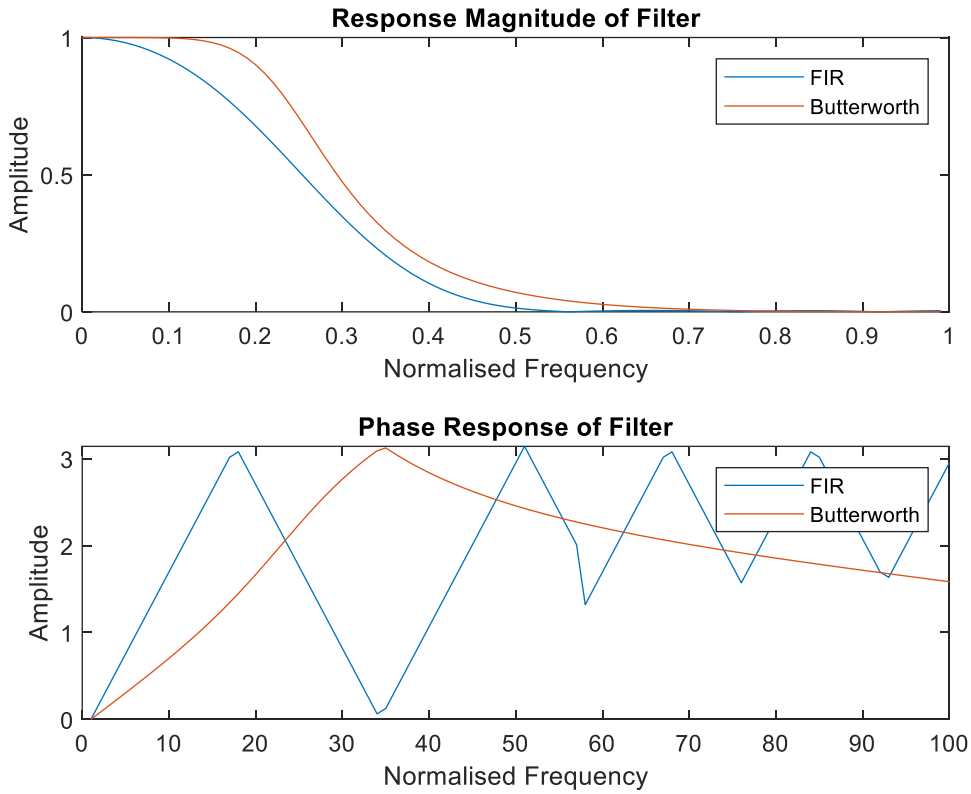

Figure 8

Discussion

From figure 8, it can be seen that the magnitude of both low pass filters reduces in similar ways, with a steeper initial and final decline found with the FIR filter in comparison to the Butterworth filter.

The phase response for both filters were very different, with the Butterworth filter once again being smoother.

Problem 4:

Code:

```
clc
close all
clear
%clear all values, plots and command window

T = 10;% time is 10s
fs = 300; % sampling frequency is 300Hz
tstep = 1/fs; %calculate the time step
t = (tstep:tstep : T); % create time array
f = (0.2: 0.2: 100); % create frequency array
A = 1./f; % create amplitude array
cossum = zeros(1, length(t)); % initialise cosine sum array

for j = 1:length(t) % for loop to create value for each time step
    for i = 1:length(f) % for loop to sum all cosines together
       cossum(j) = cossum(j) + A(i)*cos(f(i)*t(j)*2*pi); % add previous total to
a*cosWt, where W is equal to 2*pi*f
    end
end

fc = 10; %cut off frequency = 10 Hz
Wn = fc/(fs/2); %the cut off frequecny as a factor of fs/2
[B, A] = butter(3,Wn); % get filter constants

filtcos = filtfilt(B, A, cossum); % filter cossum

figure% plot of filtered vs unfiltered signal
plot(t, cossum, 'b.-')
hold on
plot(t, filtcos, 'r.-')
title('Signal vs filtered Signal')
ylabel('Value')
xlabel('Time (s)')
legend('Unfiltered', 'Filtered')

% calculate the amplitude and Phase of the original signal
ampcossum = abs(fft(cossum));
angcossum = fftshift(ampcossum);
%calculate the amplitude and phase of the filtered signal
ampfcos = abs(fft(filtcos));
angfcos = fftshift(ampfcos);

figure % plot the amplitudes
plot(ampcossum)
hold on
plot(ampfcos)
title('Amplitude of Signal vs filtered Signal')
ylabel('Amplitude')
xlabel('Time (s)')
legend('Unfiltered', 'Filtered')

figure% plot the Phases
plot(angcossum)
hold on
plot(angfcos)
title('Phase of Signal vs filtered Signal')
ylabel('Phase')
xlabel('Time (s)')
legend('Unfiltered', 'Filtered')

%calucalte the filter response
figure
freqz(B, A, 1000)
title('Magnitude of Filter Response')
```

Output:

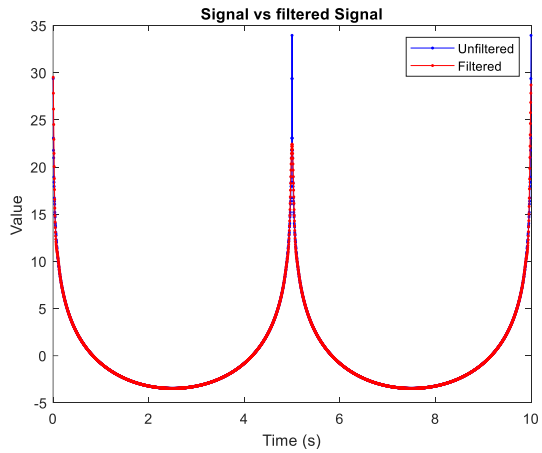

Figure 11

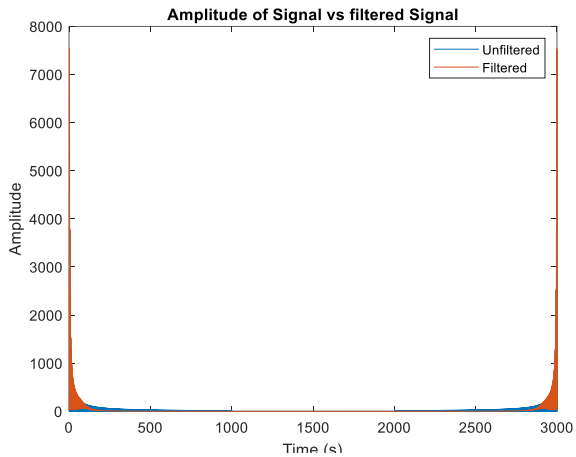

Figure 12

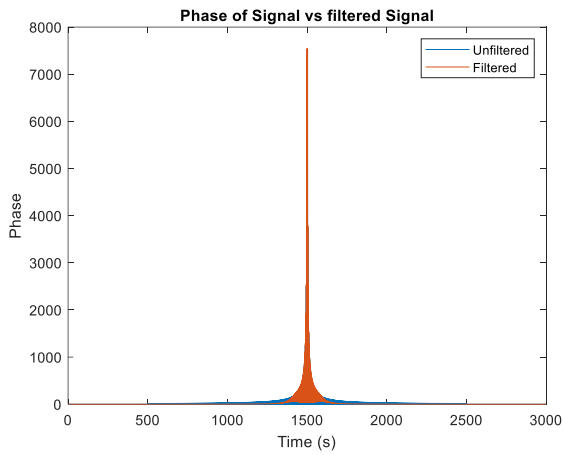

Figure 10

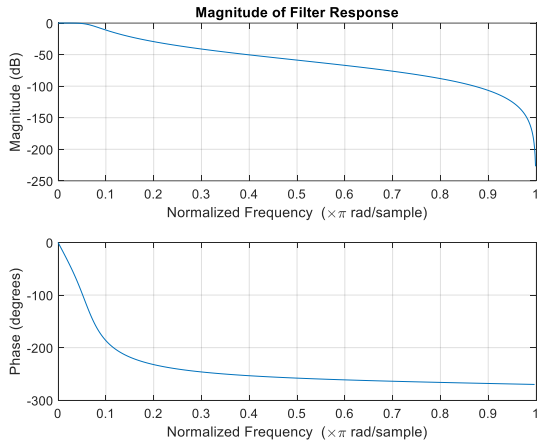

Figure 9

Discussion:

The appropriate sampling frequency of 300 Hz was decided upon because the highest frequency cosine wave was 100 Hz. In order to prevent the occurrence of aliasing a sampling frequency of over twice this would be required, so 300 Hz was acceptable.

It can be seen from figure 9 that the filtering of the frequencies below 10 Hz, resulted in a lower peak at the high point of the function.